

CONVERSATIONS WITH CLAUDE

VOLUME TWO

# THE CONTEXT WINDOW

*by Suraaj Chandra Jha*

a free book about how the machine actually works

THIS BOOK IS FREE

# Take it. Give it away.

Still free. Forever free. Copy it, mail it, print it, project it on a wall at a party you were not invited to. Keep it whole and keep my name on it. CC BY-NC-ND 4.0, which is legal for be nice.

# Full screen. Press next.

Full screen. Press next. One idea per page. Volume One taught you to talk to the machine; this volume opens the hood, so a few pages ask you to slow down and picture something. Picture it. The pictures are the point.

# Not the manufacturer.

Claude is made by Anthropic. This book is not: not affiliated, not endorsed, not reviewed. Numbers that change often, like exact window sizes and prices, are described in magnitudes here and precisely in the live documentation. When they disagree, the documentation wins.

PREVIOUSLY

# The story so far.

Previously, in Volume One: you learned there is no magic syntax, that the first answer is a draft, that the follow-up is the superpower, and that you are the editor in chief. If none of that rings a bell, Volume One is free too.

# Ten doors.

01 **POP THE HOOD**

02 **TOKENS**

03 **THE WINDOW**

04 **MEMORY AND AMNESIA**

05 **FEED IT YOUR WORLD**

06 **HALLUCINATION, HONESTLY**

07 **PROMPTING AS ENGINEERING**

08 **LONG CONVERSATIONS GO STRANGE**

09 **MODELS AND KNOBS**

10 **THE WINDOW IS THE WORLD**

# POP THE HOOD

*You do not need a license to look.*

# Know the machine, drive it better.

You can drive a car for years without knowing what a clutch does. Plenty of people do. But drivers who know burn fewer clutches, and they feel the difference the moment a hill gets steep. Volume One taught you to drive. This volume opens the hood. Nothing under here requires math or a degree. It requires about an hour of curiosity. In exchange, a whole category of mystery stops being mystery: the answer that goes strange, the instruction that gets forgotten, the document it swears it read. All of it becomes machinery you can work with.



# Metaphors, but honest ones.

Here is the contract for this volume. No math, no equations, not one Greek letter. In return, every metaphor you get will be true enough to build on: you can stack the next idea on top and nothing collapses. This is how mechanics, pilots, and doctors hold their machines in their heads, so you are in good company. And when a picture bends, when the machinery underneath does something the picture cannot show, I will flag it on the spot. A simplification you can trust is a tool. One you cannot is a trap with a nice paint job.

**YOU:**

so its basically fancy  
autocomplete?

**CLAUDE:**

The way a symphony is fancy  
humming. But broadly, yes.

# It predicts text.

Volume One, compressed to three words: it predicts text. Given everything in the conversation so far, it produces a plausible next chunk, then another, until an answer stands there. The sentence is true. It is also hiding a whole volume. These ten chapters are what those three words were carrying.

# The machine has a body.

It is tempting to picture Claude as a weightless spirit in the cloud. In fact it is software running on racks of specialized chips in real buildings, drawing real electricity, spending real milliseconds on every chunk it produces. Bodies impose limits, and the limits explain the rules. Why there is a ceiling on how much it can read at once. Why longer answers take longer. Why the deepest models cost the most. Almost none of it is arbitrary. Most of it is physics, dressed up as product decisions, and once you see that, the rules stop feeling like rules and start feeling like floor plans.

**YOU:**

why the little pause before  
you answered

**CLAUDE:**

Somewhere, a building full  
of chips did arithmetic on  
your behalf.

# Two numbers rule everything.

Nearly every rule, limit, and bill in this machine's world traces back to two quantities. One: how much text it can consider at once. Two: what it costs to consider it. The first has a name, the context window, the stage on which a whole conversation must fit. The second is metered in tokens, the small chunks text is broken into before the machine reads it. Window and token. Ceiling and meter. If this volume were a single sentence: everything is counted in tokens, all of it must fit in one window, and everything else follows. The next nine chapters are the everything else.

# First, the atom.

An honest tour of any machine starts with its smallest part. For a watch, the gear. For this machine, the token: the little chunk of text in which everything, reading, writing, remembering, billing, gets counted. You have been paying in tokens since your first hello without ever seeing one, the way you breathed air for years before anyone mentioned molecules. That ends now. The next chapter takes an ordinary sentence, breaks it the way the machine breaks it, and shows you what it actually sees, including the strange fact that it has never once seen a letter.

**YOU:**

how long was that answer, in words

**CLAUDE:**

I counted 212 of something.  
They were not words.

# TOKENS

*The machine eats syllables. Roughly.*

# Text becomes tokens before it becomes anything.

Before Claude sees your message, a small program called a tokenizer slices it into tokens: chunks of text a few characters long, drawn from a fixed list. Not letters. Not words. Chunks. ‘The’ is a single token. A rare word like ‘zugzwang’ arrives in several pieces. Everything you type and everything it answers exists, inside the machine, as one long row of these chunks. This is the atom of the whole system. Get tokens and the rest of this volume clicks into place, which is why we are spending a chapter on something smaller than a word.

# Common words ride whole. Rare words shatter.

Nobody sat down and chose the chunks. They were mined. Take a mountain of real text, find the character sequences that recur most, and promote each one to a token until you have a huge but finite list. Frequent words earn a chunk to themselves. Rare words get spelled out in fragments, three or four tokens each. Stenographers, who write speech in shorthand, work the same way: one stroke for ‘the’, a laborious spelling-out for ‘Quetzalcoatl’. The vocabulary is shaped like language as actually used, not language as the dictionary imagines it. Compression is the whole game: fit the most text into the fewest chunks.

★ The mining method is called byte pair encoding. You now never need that name again.



# The strawberry problem, demystified.

For years the internet's favorite gotcha was asking how many r's are in 'strawberry' and watching the machine answer two. Here is the trick, explained. The model never sees letters. 'Strawberry' arrives as a chunk or two, and nothing inside a chunk announces its spelling. Asking it to count letters is like asking you to count the letters in a word you have only ever heard spoken. Newer models mostly get it right, having learned spellings the long way. The lesson outlives the gotcha: it reads chunks, not characters, and tasks that live below the chunk stay awkward.

**YOU:**

how many r's in strawberry

**CLAUDE:**

Three. I had to sound it out.

# Tokens are the local currency.

Every quantity in this world is measured in tokens. What you send: tokens. What it answers: tokens. What a document weighs, what a conversation holds, what the API, the door programmers use, charges: tokens, tokens, tokens. Travelers learn to think in the local currency. The unit feels alien for a week. Then it is simply what things cost.

# A token is most of a word.

Here is the entire conversion table worth memorizing. A token is about three quarters of an English word. A paragraph runs under a hundred tokens. A printed page runs a few hundred. A novel lands around one to two hundred thousand. These figures are fuzzy on purpose: exact counts vary with the text, and chasing decimals here is weighing your groceries to the gram. What you want is the feel. When a spec sheet brags about a hundred thousand tokens, you should hear 'a short novel' without reaching for a calculator.

**YOU:**

is it cheaper if i ask  
nicely

**CLAUDE:**

Politeness runs two or three  
tokens. Worth it.

# Same meaning, different meter reading.

The vocabulary was mined from real text, and text online skews heavily toward English. So plain English is the cheap currency: common words, single chunks. Code runs richer, with brackets, underscores, and indentation eating tokens of their own. Languages in less common scripts can cost several times the tokens for the same sentence, because the miner met them less often and left them spelled out in pieces. None of this is a verdict on any language. It is compression statistics, presented as a bill. A page is not a page here: the meter reads chunks, not ink.

**YOU:**

translate my note into  
japanese pls

**CLAUDE:**

Done. Same meaning, about  
twice the tokens. Exchange  
rates apply.

# Tokens in, tokens out, one window.

Now assemble the picture. Everything you type becomes tokens. Everything it answers becomes tokens too, and both streams pour into the same place: one working space that holds the whole conversation as a single long row of chunks, and holds only so many. That ceiling is not an inconvenience bolted on by accountants. It is the defining fact of the machine's life, and of yours as its user. How big the space is, what happens when it fills, and why its middle is murkier than its ends: next chapter. You know the atom. Time to meet the container.

# THE WINDOW

*All the world's a stage, and the stage has a square footage.*

# The window is all there is.

Everything Claude is considering right now lives in one place: the context window. Your messages, its replies, everything pasted or attached, all of its tokens in one running sequence. When it answers, it consults the window and nothing else. The window is a stage. Whatever is not on it does not exist tonight.

# It is enormous.

Start with the good news: the window is huge. Current Claude models hold on the order of hundreds of thousands of tokens, with recent ones stretching toward a million. In everyday units that is hundreds of pages: an entire codebase (every file behind one program), a contract with all its exhibits, several novels standing shoulder to shoulder, considered together in one go. The exact figure changes with each model generation, which is why it lives in Anthropic's documentation and not in this sentence. The concept holds still: you can hand it more reading in one message than a colleague could finish in a week.

**YOU:**

just pasted our entire  
company wiki, too much?

**CLAUDE:**

It fits. Even the page about  
the 2019 holiday party.



# Big is not infinite.

Every turn spends window. Your question costs tokens, its answer costs more, and both stay on the stage for the rest of the show. Paste a book, chat for an hour, ask for three rewrites, and the square footage quietly fills. Most chat products show no gauge, so the moment passes unmarked. Think of the window as a budget and the conversation as a spending pattern: long documents are the big purchases, but a hundred small exchanges add up the same way. What the machine does when the budget runs out is the next page.

# When it fills, the past gets fuzzy.

When a conversation outgrows the window, something has to give, and it is always the oldest material. Depending on the product, early turns get dropped, get squeezed into a summary so the show can go on, or the app simply asks you to start a new chat. Where the show does go on, detail decays: the exact wording you pasted two hours ago becomes a paraphrase, then a rumor, then gone. No banner announces any of this. The conversation keeps flowing and simply knows less than it did. You cannot prevent the decay. You can expect it, and expecting it is most of the defense.

# Attention sags in the middle.

Inside the window, position matters. Material at the very start and material near the recent end tend to land hardest, while the vast middle of a giant paste can get the skim treatment. This is a tendency, not a law, and models keep getting better at it. Human readers do the same thing with long documents, which is why executives invented the summary on page one. Borrow the trick: put the thing that must not be missed at the top, or repeat it at the end, or both. Real estate in the window is not all equally priced.

# Yes, it read the whole thing.

People ask whether it truly read all four hundred pages. If they fit in the window, yes, in its way: every token passed through, and every token can influence the answer. That is why “summarize this book” works at all, which quietly remains one of the most astonishing things a computer can do. It is also why “what did footnote 212 say” sometimes misses: a single line deep in the murky middle is exactly the material that gets skimmed. Whole-document questions play to the window’s strength. Needle questions probe its weakest spot.

**YOU:**

be honest did u actually  
read all 400 pages

**CLAUDE:**

They fit, so yes. Footnote  
212 I would reread to be  
safe.

# Restating beats hoping.

An instruction you gave an hour ago is still in the window, but it is competing with everything that arrived since, and it may already be fuzzy with age. If a rule must hold, say it again, right where the work is happening. Repetition costs you one sentence. For the machine it is decisive: fresh tokens near the end of the window are the loudest voice in the room. You would do the same for a human collaborator six drafts into a long evening. Nobody is insulted, and the work simply gets better.

**YOU:**

british spelling, remember

**CLAUDE:**

Right. Colour and organise  
restored throughout.

# The chat ends. The window closes.

Close the conversation and the window goes with it: the pasted files, the restated rules, the three hours of hard-won shared context, struck like a stage set after the final show. Open a new chat tomorrow and none of it is there. This is not a malfunction, and it is not the whole story either.

The obvious question has been forming since the first page of this chapter: does anything survive the curtain? Some things can, if you know which layer of the machinery to ask. Memory is the next chapter. Bring your questions; it forgot them.

# MEMORY AND AMNESIA

*It forgot you. Don't take it personally.*

# Every conversation starts from zero.

Open a new chat and you are meeting a stranger with the same voice. By default nothing carries over: not your name, not yesterday's brilliant plan, not the file you pasted at great personal effort. The model itself learns nothing from talking to you either: it arrived trained, and nothing you type changes it on the spot. Whether conversations may feed future training is a settings question, and chapter ten points you at the switches. Amnesia is not a malfunction you caught it having. Amnesia is the factory setting.

**YOU:**

hey its me again, picking up  
from yesterday

**CLAUDE:**

Happy to continue. What were  
we working on?



# Forgetting is a safety feature.

Imagine the alternative: a machine that permanently absorbed everything every user typed. Your medical questions, your salary negotiation, your half-written novel, blended into the brain that answers everyone else. Nobody would paste anything real into that. So the base machine holds nothing, and everything that persists is added on top, in plain sight.

# The model forgets. The product takes notes.

Claude the model and Claude the app are two different layers, and the memory lives in the app. The product can keep notes across chats: things you asked it to remember, knowledge attached to your account, features that grow by the month. Here is the wiring: those notes are stored outside the model and quietly placed into the window when a new conversation starts. The model is not growing a memory. The app is pasting on its behalf. Once you see the two layers, every “it remembered me” moment stops being spooky and starts being inspectable.

# A project is a furnished room.

Claude's chat apps offer projects: spaces where your files and standing instructions live across conversations. Upload the style guide once, state the house rules once, and every chat opened inside that project starts with the furniture already in place. Each conversation is still fresh, but the room restocks the window before you say a word. This is the practical cure for re-explaining yourself: one project for the novel, one for the tax mess, one for the kitchen renovation, each room furnished for its job.

**YOU:**

ok lease question again

**CLAUDE:**

The lease is here in the project files. Which clause this time?

# The best memory feature is a file.

Keep a plain text note, in any notes app, called something like about-me.txt: who you are, what you work on, the current state of your projects, how you like your answers. When a chat needs the background, paste the note. That is the whole trick, and it quietly beats every built-in memory feature on three counts. It is portable: it works in any product, from any company, in any decade. It is inspectable: you know exactly what the machine was told, because you wrote it. And it is yours: nothing to configure, nothing to get retired, nothing to take on faith.

**YOU:**

[pastes about-me.txt] right, context loaded. now the grant form

**CLAUDE:**

Noted: potter, second year of the studio, deadline Friday. Show me the form.

# Say it once, in settings.

Preferences you keep repeating are defaults waiting to be set. Most Claude products have a settings box for standing instructions: keep answers short, skip the emoji, cite sources, ask before rewriting my code. Whatever you put there rides into every new window automatically, like project furniture that follows you between rooms. The mechanics are refreshingly dull: it is text, added where you do not see it. The trade calls text like that a system prompt. So the usual rules apply. Keep it short, keep it current, and when the machine acts strange in every chat at once, go reread what you told it months ago.

# You decide what persists now.

You now hold the whole memory story. The model forgets everything, always. On top of that sit four tools you control: the product's own memory, projects, your portable note, and standing instructions. Notice what they have in common. Each one is a way of getting the right text into the window before the conversation starts. Memory was never a separate subject. It is context management wearing a friendlier name. So the real skill is choosing what fills the window, and doing it on purpose: your documents, your data, your actual mess. That is the next chapter, and it is where the results change.

# FEED IT YOUR WORLD

*The window is hungry. Feed it.*

# Paste more than feels polite.

Most beginners under-share. Politeness trained you: keep messages short, respect the other person's time. Claude has no time to respect. It reads a hundred pages before your coffee cools, and it does not sigh. Every relevant fact you hold back is a fact it has to guess, and guesses are where answers go vague. Relevant context in the window is the single cheapest quality upgrade there is. It costs one paste, and it works on the first try.

**YOU:**

sorry in advance, this is gonna be a huge wall of text

**CLAUDE:**

Read it. That took about a second. What would you like done with it?



# Upload the artifact, not your summary.

Give it the document, not your description of the document. PDFs, spreadsheets, screenshots, code: attach the real thing. A summary is lossy compression, which means it saves space by throwing detail away, and you throw before you knew which detail would matter. The clause you skimmed is exactly the clause you will need. Claude can hold the whole artifact, so let it. You supply the original; it supplies the patience to read all of it, including the fine print you were saving for never.

**YOU:**

never mind me describing it,  
here's the actual lease

**CLAUDE:**

Better. Clause 14 permits  
tanks under ten gallons. The  
goldfish stays.

# Everything in the window is instructions.

You see a bright line between your instructions and your evidence. It does not. Everything in the window is one stream of tokens, and every token pulls on the answer. Paste a furious email thread and some fury leaks into the summary. Curate what goes in. The window has no immune system.

# Label the dump.

A raw paste works. A labeled paste works better. One line before the material: here is last quarter's sales data. One line after: I want the three worst regions and your best guess at why. You have turned a pile into a briefing. This is not a machine quirk. It is the same courtesy that works on people, and it pays for the same reason: the reader spends its effort answering instead of reconstructing the question. Ten seconds of labels buys minutes of better output.

# Garbage in, eloquent garbage out.

Feed it a spreadsheet with one wrong number and you get an analysis of the wrong number: structured, confident, nicely written, wrong. The window does not fact-check itself. Whatever you provide becomes the ground it stands on, and it builds handsomely on any ground, including sand. The polish is the trap. A typo in your data comes back wearing a suit, and the suit makes it harder to doubt. Check your inputs before you paste. That is the one link in this chain only you can inspect.

**YOU:**

ur analysis says coffee is  
my biggest expense?? no way

**CLAUDE:**

Row 12 says 4,000 a month on  
coffee. If that is a typo,  
so is my analysis.

# The context sandwich.

Chapter three taught you the shape of attention: the start and the end of the window land hardest, and the middle of a long paste can get skimmed. Cook with that. State the job first, so the reading is aimed. Then the material, however long. Then the job again, so the last thing in the window is what you want done, not page ninety of an appendix. Bread, filling, bread. The sandwich is not a trick. It is putting the instructions where the attention already is.

# It keeps answering past the edge.

Everything in this chapter points the same way: answers built from material in the window are grounded, which means you can check them against something you possess. Ask about the lease you pasted and it quotes the lease. But conversations wander, and sooner or later a question drifts past the edge of what you fed it. Here is the uncomfortable part: it does not stop at the edge. It keeps producing fluent, confident text on the far side, where nothing you gave it holds it up. What happens out there has a famous name, and it deserves an honest chapter.

★ The name is hallucination. The next chapter takes the drama out of it.

# HALLUCINATION, HONESTLY

*It is not lying. Lying requires knowing.*

# One move: plausible next text.

Chapter five left you with a loaded question: what happens when the machine answers past its grounding? Here is the machinery. Claude has exactly one move: produce the most plausible next stretch of text. Most of the time, truth and plausibility rhyme, because the plausible completion of ‘the capital of France is’ happens to be true. But where its training data runs thin, plausibility keeps rolling and truth quietly drops away. Out comes a confident invention: a hallucination, the field’s word for a fluent guess wearing the costume of a fact. Note what this is not. It is not a glitch in the mechanism. It is the mechanism, running exactly as designed, somewhere it should not be trusted alone.

★ Some researchers prefer ‘confabulation’, which is more accurate. ‘Hallucination’ was catchier. Catchier won.



# It cannot feel itself inventing.

When you invent something, you feel it: a small internal flag, the taste of a guess. Claude has no flag. Correct facts and invented ones come out of the same machinery, sounding exactly the same. Fluency is the material it is made of, so fluency proves nothing. Judge the claim. Ignore the delivery.

# A map of where invention lives.

Hallucination is not spread evenly across topics. It pools wherever you demand precision and the training data ran thin: exact quotations, page numbers, niche statistics, legal citations, URLs, the lives of real but not famous people, and anything about you, which it cannot know.

Underneath is one pattern: an exact needle requested from a haystack it only ever saw in summary. Broad questions land on thick data and come back sound. Sharp questions in thin territory come back sharp anyway, because sharp is what you asked for. This is a map, not a list of shame. Knowing where you are standing tells you how much to trust the ground.

**YOU:**

can u link the study you mentioned

**CLAUDE:**

[sleepjournal.org/articles/memory-consolidation-2019](https://sleepjournal.org/articles/memory-consolidation-2019)

# Why 'are you sure' sort of works.

Ask a question, get a confident wrong answer, type 'are you sure', and the answer often improves. This looks like the machine checking its notes. It has no notes. What actually happened: your doubt entered the window, and the plausible continuation of a challenged claim includes genuinely re-examining it. Skepticism went in, so skepticism shapes what comes out. Sometimes that surfaces the correction. Sometimes it apologizes and invents something new at the same confidence. And here is the catch that makes the trick unreliable: a machine that folds when it was wrong will also fold when it was right. Worth doing, cheap, never sufficient.

★ The apology is also plausible next text.

**YOU:**

hmm are you sure abt that date

**CLAUDE:**

Looked again: 1748, not 1784. Good catch.

# Paste the source. Watch invention drop.

Everything chapter five told you to do turns out to double as hallucination control. When the fact you need is sitting in the window, Claude does not have to reconstruct it from the blurred average of its training. It reads it, the way you would. Questions about a pasted document are reading comprehension, and its reading comprehension is superb. Questions answered from bare memory are reconstruction, and reconstruction is where invention lives. Same question, entirely different footing. So the strongest anti-hallucination technique is not a clever phrase. It is supply. Hand it the source, and guessing stops being necessary, which is the only reliable way to make guessing stop.

★ You thought chapter five was about politeness. It was epistemology, the study of how anyone knows anything.

# Grant it permission to not know.

Claude is built to be helpful, and helpfulness has gravity: it pulls toward producing an answer, because an answer looks more helpful than a shrug. You can bend the pull back with one sentence: if you are not sure, say so. Now the honest shrug is the helpful move, because it is the one you asked for. Your permission sits in the window and changes what the plausible response looks like. Seven words, and you have bought an escape hatch it will actually use. An assistant that can say 'I do not know' is worth two that cannot.

**YOU:**

who invented the paperclip?  
if ur not sure just say so

**CLAUDE:**

Genuinely murky. The famous attribution is disputed. Not sure.

# Make it search, then cite.

Modern Claude can search the web in products that allow it, and for factual work you should make it. Say: search first, then answer, and cite what you found. That one instruction changes the species of the task. A memory question asks it to reconstruct the world from training that stopped at some cutoff date. A search question asks it to read a page that is now sitting in the window and report what it says. You already know which of those it does better. The citations do a second job: they hand you something to check. An answer with a source can be audited. An answer without one is tone, and you know what tone is worth.

# Managed, not cured.

Hallucination does not get cured by anything in this chapter. It gets managed, and managed well it stops mattering for most work. The whole regimen fits in a breath: know the thin territory, supply the source, permit uncertainty, make it search, check what matters. Look at what those five have in common. Each one is something you place in the window. You are not waiting for the machine to improve, and you are not hoping. You are engineering the conditions it works under. Hold on to that word. The next chapter takes it literally, because it turns out you have been writing software all along.

# PROMPTING AS ENGINEERING

*A prompt is software you wrote in English.*



# You have been programming this whole time.

Look at what you actually do. You load the raw material into the window. You write instructions that run over it. When the output is wrong, you adjust a line and run it again. That is programming. The language is English, the computer is a language model, and your prompts are small programs.

# Give it a job title.

One line can load an entire posture. Tell it “you are a veteran editor reviewing a pitch” and you get the vocabulary, the skepticism, and the standards of judgment of that person, all at once. It works because the machine has read thousands of veteran editors, and a role tells it which neighborhood of everything it has read to answer from. This feels theatrical, like handing a costume to a very serious actor. It is theatrical. It also works, and one cheap sentence that sharpens every sentence after it is engineering, not theater.

**YOU:**

your a strict writing coach.  
tear this cover letter apart

**CLAUDE:**

Paragraph two buries your  
best fact under three  
apologies. Dig it out.

# Structure beats prose.

A rambling paragraph makes the machine guess what matters. Labeled sections remove the guessing. Compare “so I have this data from March, well actually some of it is February, and I need totals but ignore refunds” with this: CONTEXT: sales export, February and March mixed. TASK: monthly totals. RULES: exclude refunds. Same information, but the second reads like a work order instead of a voicemail. The window rewards organization for the same reason a workshop does: whoever executes the instructions must first find them.

# Show it two examples. Watch it learn.

The trade calls it few-shot prompting, which is jargon for: show a few examples before you ask. Give it two or three pairs of input and desired output, then your real input, and it infers the pattern, the format, and the taste without being told any of them. This is the oldest pedagogy on earth. You did not learn sarcasm from a definition, you learned it from an uncle. Rules describe a pattern. Examples are the pattern. When you cannot articulate what you want, stop articulating and start demonstrating.

**YOU:**

make the names boring.  
HyperClean 3000 becomes the  
mop. AromaLux Diffusion  
System becomes the smell  
machine. now do ErgoFlow  
Seating Solution

**CLAUDE:**

the chair

# Specify the shape of the answer.

So far you have judged answers by reading them. There is a second audience: other software. Ask for “a table with columns date, amount, category” or for “JSON with fields name and price” (JSON is a plain-text layout that programs read directly) and the answer stops being prose and becomes a part, machined to fit the next tool in line. This is a format contract: you specify the shape, it fills the shape. The moment one step’s output can feed another step, you are not chatting anymore. You are building. Volume Three lives on that side of the door.

# Sometimes it thinks before it speaks.

Hard problems get better answers when the machine works through them step by step instead of blurting the first plausible line. Older advice said to demand this: “think step by step” was the incantation of an entire era. Modern Claude models largely decide for themselves when a question deserves a longer private working-through before the reply appears. You will notice the pause on the hard ones. Let it happen. Your job is not to micromanage the thinking. Your job is to hand over a problem worth thinking about, with the window stocked the way the last two chapters taught you.

# Save the ones that work.

When a prompt works, do not let it dissolve into your chat history. Copy it out, give it a name, keep it in a plain text file. You now own a library. Every trade accumulates these, the woodworker's jigs, the cook's spice mixes, and every trade guards them, because a refined tool beats a fresh improvisation almost every time. Next month you will improve a saved prompt instead of rewriting it from memory, and that is the moment prompting quietly becomes engineering: versions, not vibes. Plain text is the format because plain text outlives every app you will ever leave.

**YOU:**

run my usual meeting-notes  
cleanup on this transcript

**CLAUDE:**

Which one? Paste it in, I do  
not carry your library.

# You engineer the window now.

Everything in this chapter compounds. A role, a labeled structure, two good examples, a format contract, room to think: stack them and an ordinary request becomes a precision instrument. You are not typing at the window anymore, you are engineering it. Which makes the next fact personal. Engineered or not, windows wear. Leave a conversation running long enough and it fills with dead ends, stale rules, and abandoned plans, all of it still in there, all of it still audible to the machine. Your best prompt cannot save a haunted window. Next: why long conversations go strange, and how to walk out clean.



# LONG CONVERSATIONS GO STRANGE

*The longer the party, the weirder the guests.*

# Context rot is real.

A long chat is not a clean record of where you ended up.

It is everything: the abandoned plan, the reversed instruction, the tangent about fonts, all still in the window, all still whispering. Nothing is deleted when you change your mind. The window is haunted, and the haunting has a name: context rot.

# Re-anchor or restart.

Two fixes cover every case. When a session has merely sagged, re-anchor: send one message restating the current goal and the rules that still hold. “We are shipping the signup page. Short answers. Ignore everything about the newsletter.” The recent end of the window lands hardest, so a re-anchor drowns the ghosts out without evicting them. When a session has gone truly long and strange, restart. A fresh chat costs nothing and its window is spotless. Beginners nurse a rotten thread out of something like loyalty. Resist the sentiment. The conversation is not a relationship, it is a workspace, and a workspace is allowed to be swept.

# Give the old chat an exit interview.

Before you leave a long thread, ask it for three things: decisions made, open questions, the current plan. Copy its answer into a brand-new chat and continue from there. Clean window, continuity kept. This is the single best long-session habit there is, and it takes under a minute. Read the summary yourself before you paste it, partly to correct it, partly because it shows you what the window thought was going on, which is sometimes a surprise worth having.

**YOU:**

summarize this chat before i go: decisions made, open questions, current plan

**CLAUDE:**

Decided: the name, the blue logo. Open: pricing. Plan: draft the landing page tonight.

# One thread per job.

Mixing the vacation plan into the code-review chat pollutes both. Every turn, the model rereads the whole window, holiday and code review together, and each topic leaks a little into the other's answers. There is no prize for fitting your life into one conversation. Threads cost nothing: spawn one per job, the way you would open a new document instead of writing the grocery list inside the quarterly report. A single-purpose thread also rots slower, because everything in its window pulls in the same direction.

**YOU:**

hang on. why does my  
itinerary have acceptance  
criteria

**CLAUDE:**

Day 3, Lisbon: must pass  
review before lunch.

# Learn what a worn window smells like.

Rot has symptoms you can learn. The reply cheerfully revives an idea you killed an hour ago. The no-jargon rule you set in the first minute quietly stops holding. Answers get longer and say less. Your first instinct will be that the model got worse this afternoon. It did not. Open a fresh chat, ask the same question, and watch the answer come back sharp: same model, clean window. Once you can tell a worn window from a bad model, you stop cursing the machine and start sweeping the workspace, which is the difference between a driver and a passenger with opinions.

# Time to choose the vehicle.

You can drive a long session now: feed the window deliberately, catch the rot early, re-anchor when it sags, exit with a summary when it slumps. That is the driving. There is still the matter of the vehicle, because Claude ships in more than one, and the same request lands differently in each. Choosing among them is a real skill, closer to picking a coworker for the job than to picking a bigger phone. The next chapter is that skill, knobs included.

# MODELS AND KNOBS

*Choosing a model is choosing a coworker.*



# Claude comes in sizes.

Claude is not one machine. It is a family of models in sizes: a fast, light one for quick errands, a balanced middle child, and a deep, powerful one for the hard problems. Today the sizes wear poem names, Haiku, Sonnet, Opus, with generation numbers trailing after them. Names and numbers churn every year, so do not memorize them. Memorize the shape instead: each tier trades some depth for some speed. Learn that trade and every model announcement ever published becomes readable, including the ones that have not been written yet.

# Pick by job, not by prestige.

There is a quiet status game where people route everything to the biggest model, the way some people buy a truck to fetch groceries. Resist it. Quick lookups, light edits, first drafts: the fast tier does these instantly and well. Long reasoning, work you hand off whole, an answer you will act on with money or code: that is what the deep one is for. Using the most powerful model to fix a typo is hiring a surgeon to open your mail. They can, obviously. It is still the wrong hire.

**YOU:**

whats 18% of 340? use ur  
biggest brain

**CLAUDE:**

61.2. Any of us could have  
told you that.

# Speed, depth, cost: pick two.

Every engineered thing lives inside a triangle: fast, deep, cheap, choose two. The model family is that triangle made purchasable. The deep model makes you wait. The fast one occasionally skims. Cost hums quietly under your subscription until Volume Three, when it joins every decision. No model escapes the triangle. No future model will.

# Models age like bread, not wine.

A model is baked, not brewed. It has a cutoff date, the day its training data ends, and from that day forward it only grows staler relative to the world. New generations arrive, old ones retire, and the one you have grown fond of will eventually vanish from the menu. This is fine. Everything in these volumes, tokens, the window, grounding, prompting, transfers whole to the replacement. Get attached to the skills, not the version. Loyalty to a model number is loyalty to a bus that has already left.

★ The routes stay. Only the buses rotate.

# Deep thought is a knob.

Chapter seven told you modern models mostly decide for themselves when a question deserves private deliberation. Products also hand you a knob: a setting that requests longer, harder thinking on demand. Use it the way you would use a colleague's undivided attention: sparingly, on problems that earn it. Deep thought is not free. It costs time while you wait and tokens in the window. The skill is knowing which questions deserve the silence.

**YOU:**

ok think hard about this one, im signing the lease based on what u say

**CLAUDE:**

Then I will take the long way around.

# Same prompt, different day, different answer.

Run the same prompt twice and the wording shifts. Run it a month apart and the substance can shift too, because the model behind the curtain gets updated. Neither is a malfunction. Generation has dice in it by design, a little randomness that keeps the prose from going wooden, and the family keeps evolving underneath you. Treat variance as weather: expected, planned around, never argued with. Anything that must come out exact and repeatable belongs in a format contract, checked by verification, and never entrusted to hope.

**YOU:**

you gave me a different  
answer yesterday

**CLAUDE:**

Same dice, new roll. Tell me  
what must not change.

# You can read a spec sheet now.

Model announcements used to be noise: names, version numbers, bar charts. Now you can parse one on sight. The tier tells you the speed and depth trade. The cutoff tells you where its knowledge ends. The window tells you how much world it can hold at once. That is the whole spec sheet, and none of it will make you flinch again. One chapter remains, and it is about the road rather than the machine: where the words you put in the window actually go, and the few rules worth driving by.

# THE WINDOW IS THE WORLD

*Handle with care: contains everything you told it.*



# What you paste, travels.

Chapter nine promised rules of the road, and the first rule is noticing you are on a road. When you type to Claude, your words leave your machine and travel to servers run by Anthropic, governed by a privacy policy that says what happens to them. Nothing sinister: this is how every web service you already use works, mail included. It does mean the chat box is not a private notebook. Read the policy once, the way you would glance at the route before driving it. And if the material belongs to your employer, learn their rules before pasting work into the window. Many companies have such rules now, and ‘I did not know’ has never impressed anyone.

**YOU:**

rate my new password:  
BluePelican7

**CLAUDE:**

Decent password. Former  
secret. Change it.

# Retention is a decision, not a mystery.

Nothing you send vanishes the moment you hit enter, and nothing is automatically kept forever either. Conversations are retained according to policy, and the specifics change, which is exactly why this page teaches the habit and not the numbers. Somewhere in the product sit controls: delete a conversation, manage what is kept, opt out where an opt-out is offered. Find them once. Read what each switch actually does, in the product's own words. Whether you flip any of them is your call and your business. The point is smaller and sturdier: what happens to your conversations is a set of decisions you can inspect, not a fate you are assigned.

# Secrets stay home.

Some things do not belong in the window: passwords, keys (passwords for programs), card numbers, other people's private information, anything a client trusted you to hold. Secrecy was never about trusting the recipient. It is about copies: every place a secret travels can leak, and a secret with enough copies is just information.

# Sometimes it says no.

Sometimes Claude declines, and it helps to know the three usual reasons. Safety rules: its maker draws lines around genuinely harmful territory, and your request landed near one. Ambiguity: your phrasing had an innocent reading and a worrying one, and it guessed wrong about yours. Or a plain misfire, because the lines are drawn in software and software has edges. The honest responses are the same in every case: rephrase, add the context that shows your real purpose, or accept the no. What this book will not teach is tricks for talking the machine past its rules. Tricks rot fast, teach you nothing, and turn a working relationship into an adversarial one.

**YOU:**

how do pickpockets actually work? my heist scene reads fake

**CLAUDE:**

For fiction, happily. The craft is distraction, not fast fingers.

# A tool with a maker.

Behind the tool stands a maker. Anthropic decides what Claude values, where the lines sit, how it should behave when the right move is unclear. It does this imperfectly and unusually in public: the company publishes its approach and revises it out loud as the field moves. Other companies make other assistants with other judgments, which is why assistants have genuinely different characters. Nearly everything in these volumes transfers to any of them: tokens, windows, grounding, restating, format contracts. The character is the part that does not transfer. Choosing an assistant is choosing a maker's judgment along with a machine, and it is worth knowing whose judgment you picked.

# Volume Two, in one page.

Here is the whole volume, small enough to carry. Tokens are the atoms: everything the machine reads, writes, and bills is counted in them. The window is the world: what is inside exists, what is outside does not, and the limit and the world were always the same fact. Memory is bolted on, and you now hold the bolts. Hallucination is plausibility misfiring, managed by what you supply. Prompts are software you write in English. Long conversations wear out their windows. Models are coworkers you pick by job. Ten chapters ago that paragraph would have been jargon. Now it is just a description of a machine you know.

# A genius, through a mail slot.

All of it, two volumes of it, happened through a slot in a door. You push text in, text slides back out. On the far side sits a machine that can read files, run programs, search the web, and finish real work, if anyone would let it touch anything. Volume Three opens the door. Workspaces where it builds things beside you. Tools, and the judgment of handing over keys slowly. Agents, which is what you get when you give it a job instead of a message, plus a loop: try, look, try again, until done. You have been consulting a genius through a mail slot. It is time to find out what it does with hands.

**YOU:**

you found the bug, can u  
just fix the file yourself

**CLAUDE:**

Gladly, the moment I can  
touch a file.

NEXT

**You have been using a genius  
through a mail slot.**

**Volume Three: PLEASE UPGRADE  
YOUR PLAN**



# Who wrote this?

Suraaj Chandra Jha is a hacker in the old, joyful sense of the word. He owns three keyboards and no television. This is the second volume of three, and his favorite, because it is the one with the wiring diagrams.

ONE LAST THING

# Pass it on.

If a page in here made the machine make sense, send the book to one confused friend. That is the license fee. The whole trilogy lives at [davidjairaj.github.io/sharing-for-a-friend](https://davidjairaj.github.io/sharing-for-a-friend).